SENTINELS

Complete Project Documentation

An AI-powered, passive website & GitHub-repository security auditor

---

This document explains, in full, how Sentinels works: every scanning agent, the scoring and checklist logic, the AI layer, the data model, the API surface, the frontend, and how the project was built across three major versions. A separate final section answers the standard pitch-deck questions (problem, solution, market, competitors) for presentation use.

Prepared for internal use — for presentation preparation and complete system understanding.

Contents

Part 1

What Sentinels Is

The one-paragraph pitch

Sentinels is a security auditor for people who are not security experts. You give it a website URL or a public GitHub repository link. Within roughly a minute, it returns a 0–100 security score, an A–F grade, and a plain-English report explaining exactly what is wrong, why it matters, and how to fix it — with an AI layer that can generate an actual code-level fix on request. It never attacks anything: every check is something a normal browser or a curious visitor could already see.

The passive-only ethic

This is the project's one non-negotiable rule, and it shapes every design decision in the system. Every check Sentinels performs is one of: reading response headers, inspecting a TLS certificate the server already presents, reading public DNS records, or making an ordinary GET request to a small number of publicly-known paths (like `/robots.txt` or `/.env`). There is no SQL injection, no brute forcing, no fuzzing, no automated form submission, no denial-of-service traffic — ever. On the repository side, the same ethic applies to code: Sentinels reads bytes out of a downloaded repository archive and never imports, installs, or executes a single line of it. The comparison the project's own documentation uses: this is like reading a building's fire-safety certificate, not testing the fire alarm yourself.

Two things you can scan

- **A live website (URL scan)** — five agents inspect the site's headers, TLS setup, DNS/email security, and whether it leaks sensitive files or a CMS fingerprint.
- **A public GitHub repository (repo scan)** — five different agents download and read the repository's source, looking for committed secrets, vulnerable dependencies, risky config, risky code patterns, and general project hygiene.

Why both scoring and grading are deterministic

The score and grade are pure, rule-based arithmetic — no model is ever in the loop for scoring. The same site or repository, scanned twice, always produces exactly the same score. This matters because it makes Sentinels trustworthy in a way a purely AI-judged tool cannot be: you can point at a specific number and know it will not silently drift between two runs. The AI layer only ever *enriches* the deterministic result — a plain-language summary, an on-demand fix suggestion, a chat window to ask follow-up questions — and if the AI is unavailable for any reason, the report is still complete.

Tech stack at a glance

Layer	Technology
Backend	Python, FastAPI, httpx, asyncio.gather / asyncio.as_completed
Live progress	Server-Sent Events (SSE)
PDF export	Playwright (headless Chromium)

Layer	Technology
Frontend	Next.js (App Router), React, Tailwind CSS
AI	Groq's free-tier API, model openai/gpt-oss-20b
Persistence	SQLite via stdlib sqlite3, no ORM
Scoring	Pure functions — deterministic, no model in the loop

The high-level workflow

Both scan types follow the exact same shape:

- **1. Input.** A URL or a GitHub repo link comes in through the landing page.
- **2. Fetch.** For a website: one normalized HTTP client is shared by every agent. For a repo: the repository is downloaded as a tarball, safety-checked (size caps, path-traversal protection), extracted to a temporary folder, and walked once into a shared file list.
- **3. Scan, concurrently.** All five agents for that scan type run at the same time (`asyncio.gather`), each producing a list of findings. If live progress is being streamed to the browser, the same agents run instead under `asyncio.as_completed`, so each one's result reaches the frontend the instant it finishes rather than only when all five are done.
- **4. Score.** Every finding subtracts points from a starting score of 100, by severity.
- **5. Checklist.** A separate rule-based system turns the same findings into a three-tier deployment-readiness checklist, plus a readiness percentage and a ready / caution / blocked verdict.
- **6. AI summary.** If a Groq API key is configured, an LLM call turns the findings into a short plain-English paragraph naming the worst real problem. If not, this field is simply empty and everything else still works.
- **7. Persist and return.** The finished report is written to SQLite and handed back to the browser, which now has a permanent link to it — refreshing the page, or coming back tomorrow, shows the exact same report.

Part 2

The Scanning Engine — Website Agents

Every website agent inherits from one shared base class, `BaseAgent`, which is also where the project's most important safety rule for agents is enforced in exactly one place: an agent must never crash the whole scan. `BaseAgent.run()` times the agent, catches absolutely any exception it might raise, and returns a normal result with an `error` field set instead — so one broken check never takes the other four down with it. All five agents share a single `httpx.AsyncClient` rather than opening their own connections.

1. Headers Agent

Category: Headers **OWASP tie-in:** A05:2021 — Security Misconfiguration

Makes one GET request and inspects four security-relevant response headers — the single most common and cheapest security signal a website exposes.

Header checked	If missing	If present
Content-Security-Policy	High — blocks XSS/injection defenses	Info / Pass
Strict-Transport-Security (HSTS)	High — allows downgrade attacks	Info / Pass
X-Content-Type-Options	Medium — allows MIME-sniffing attacks	Info / Pass
X-Frame-Options	Medium — allows clickjacking	Info / Pass

2. Recon Agent

Category: Recon **OWASP tie-in:** A05:2021 — Security Misconfiguration

Looks for information a site is unintentionally handing to an attacker before they've done anything at all.

Check	What it looks for	Severity
Generator meta tag	A `<meta name="generator">` tag revealing the CMS/framework and version	Low / Warn
robots.txt sensitive paths	Disallow entries containing words like admin, login, backup, config, .env, private — robots.txt doesn't block access, it just tells crawlers (and attackers) where to look	Low / Warn

3. TLS Agent

Category: TLS **OWASP tie-in:** A02:2021 — Cryptographic Failures

The only agent that isn't a plain HTTP call — it opens a raw socket and performs an actual TLS handshake to inspect the certificate the server presents, run off the main event loop since Python's ssl/socket libraries are blocking.

Check	Trigger	Severity
Not using HTTPS at all	Site only serves plain HTTP	Critical / Fail
Certificate invalid	Expired, untrusted, or hostname mismatch	Critical / Fail
Certificate expiring soon	≤ 30 days remaining	Medium / Warn
Deprecated protocol negotiated	SSLv2, SSLv3, TLSv1, or TLSv1.1	High / Fail

4. Exposure Agent

Category: Exposure **OWASP tie-in:** A05:2021 — Security Misconfiguration

Checks two well-known paths that, if publicly reachable, hand over real secrets or source history. Both checks are content-verified, not just status-code — a 200 response alone isn't trusted; the body has to actually look like the real thing (redirects are deliberately not followed, so a site that redirects unknown paths to its homepage can't produce a false positive).

Path checked	What confirms it's real	Severity if exposed
/.env	Body matches a KEY=value line pattern, and isn't HTML	Critical / Fail
/.git/HEAD	Body starts with "ref:" (a real git HEAD file)	High / Fail

Critically, the actual file contents are never shown in the report — only the fact that it's exposed. This precedent (report the exposure, never the secret) directly shaped how the repo-side Secrets agent handles real secrets found in source code.

5. DNS / Email Security Agent

Category: DNS **OWASP tie-in:** A05:2021 — Security Misconfiguration

Reads two public DNS TXT records to check whether the domain's email can be spoofed — the exact same lookups any real mail server performs before deciding whether to accept a message claiming to be from this domain. Queried against Google/Cloudflare's public resolvers directly, since a machine's own configured DNS server can be unreliable for this.

Record	Meaning if missing/weak	Severity
SPF (v=spf1 …)	No list of servers authorized to send as this domain — anyone can forge mail	High / Fail if missing
SPF enforcement (-all / ~all)	"+all" or "?all" means the record exists but protects nothing	High or Medium / Warn
DMARC (v=DMARC1 …)	No policy for what to do when SPF fails, and no reports of spoofing attempts	High / Fail if missing
DMARC policy (p=quarantine/reject)	"p=none" only monitors — spoofed mail is still delivered	Medium / Warn

Part 3

The Scanning Engine — GitHub Repo Agents

The repo side mirrors the website side almost exactly on purpose — same crash-proofing contract (`BaseRepoAgent.run()`), same idea of one shared context built once and handed to every agent. The one genuinely new piece of infrastructure is `repo/fetch.py`: it downloads a repository's tarball from GitHub, enforces hard safety limits (a ~50 MB repo-size cap, a total-extracted-bytes cap, a file-count cap, a per-file size cap), and extracts it using Python's path-traversal-safe `filter="data"` mode — so a hostile repository can never turn into a disk-fill or a directory-escape problem on the machine running Sentinels. Nothing from a scanned repository is ever imported, installed, or run.

1. Hygiene Agent

Category: Hygiene **OWASP tie-in:** —

Filename- and path-shape checks — no file contents are read, just what exists and what's missing.

Check	Severity if missing
README at repo root	Low / Warn
LICENSE at repo root	Low / Warn
Lockfile matching a manifest (package-lock.json for package.json, poetry.lock for pyproject.toml, etc.)	Low / Warn
Any recognizable test files/paths	Low / Warn
Any CI config (GitHub Actions, GitLab CI, CircleCI, …)	Low / Warn
.env.example / .env.sample present	Low / Warn
Any committed file over 1 MB (up to 20 flagged)	Low / Warn, per file

2. Secrets Agent — the headline feature

Category: Secrets **OWASP tie-in:** A02:2021 — Cryptographic Failures

Three layers of detection, from most to least certain, every one of them reporting only a masked form of what it found (first four and last four characters — never the real value).

Layer	How it detects	Severity
Known provider key shapes	Regex signatures for AWS, GitHub, Groq, OpenAI, Stripe (live keys only), Google API keys, Slack tokens, and PEM private-key blocks	Critical / Fail
A committed .env-shaped file	Filename matches .env / .env.<word> (template files like .env.example are explicitly excluded)	Critical / Fail

Layer	How it detects	Severity
Generic high-entropy assignment	A line like KEY = "value" where the name looks secret-shaped AND the value has high randomness (Shannon entropy ≥ 4.3 bits/char) — requiring both is what keeps this from flagging every UUID or commit hash in the repo	High / Fail

A committed lockfile's integrity hashes are deliberately excluded from the entropy check only (they're long and random by design, and would otherwise flood the report with false positives).

3. Dependencies Agent

Category: Dependencies **OWASP tie-in:** A06:2021 — Vulnerable and Outdated Components

Parses every dependency manifest and lockfile it finds (requirements.txt, package.json and its lockfile, pyproject.toml) into exact pinned versions, then batches all of them into a single query against OSV.dev — Google's open, public vulnerability database. This is a passive read of a public database, not an attack.

Outcome	Result
A pinned version has a known CVE	High / Fail, with the real vulnerability ID(s) as evidence
OSV.dev is unreachable	Degrades to a single "couldn't verify" finding — the scan still completes, it never crashes

4. Config Agent

Category: Configuration **OWASP tie-in:** A05:2021 — Security Misconfiguration

Plain regex/line scanning of three specific files — deliberately not a full YAML/Dockerfile parser, since that would add more attack surface than it removes.

File	What it checks	Severity
.gitignore	Missing entirely (blocking-severity); or present but not covering .env / private key files / node_modules	Medium–High
Dockerfile	Floating :latest base image; a secret-shaped ENV/ARG name; no USER instruction (container runs as root)	Low–High
GitHub Actions workflows	pull_request_target trigger (a known privilege-escalation vector); third-party actions not pinned to a commit SHA	Medium–High

5. Code Patterns Agent

Category: Code Patterns **OWASP tie-in:** A02 / A03 / A05 / A08 (varies by pattern)

Scans source for eight constructs that are *indicative* of a problem, not proof of one — every single finding here is a Warn, never a Fail, and lives in the checklist's "inferred" tier for exactly that reason.

Pattern	Concern
eval(...) / exec(...)	Arbitrary code execution if the input isn't fully trusted

Pattern	Concern
subprocess ...shell=True	Shell injection risk
SQL built by string concatenation / f-string	SQL injection risk
dangerouslySetInnerHTML	Cross-site scripting (XSS) risk
verify=False	Disables TLS certificate verification entirely
DEBUG = True	Leaks stack traces/internals on error, in production
Wildcard CORS (allow_origins="*")	Any site can call this API with credentials
pickle.load(s)	Unpickling untrusted data can execute arbitrary code

Part 4

Scoring, Checklist & Deployment Readiness

The scoring formula

Deterministic and pure — no randomness, no model, not even the current time. Every scan starts at 100 points. Every finding that isn't a clean Pass subtracts points based on its severity (both Fail and Warn findings cost points; only Pass costs nothing):

Severity	Points deducted
Critical	25
High	15
Medium	8
Low	3
Info	0

The final score is clamped so it never drops below 0. The letter grade is a fixed cutoff table:

Score	Grade
90 – 100	A
80 – 89	B
70 – 79	C
60 – 69	D
Below 60	F

The three-tier checklist system

A deployment-readiness checklist exists alongside the score, because a raw 0–100 number doesn't tell a non-expert developer *what to actually do next*. Every checklist item belongs to exactly one of three tiers, and the tier is the honest answer to "how do we know this?":

- **Auto** — Sentinels observed this directly from a real finding (e.g. "is HTTPS enforced" comes straight from the TLS agent). These are the only items that count toward the readiness percentage.
- **Inferred** — a weak, passive signal that's suggestive but not conclusive (e.g. a code pattern match). Explicitly labelled as such, never presented with false confidence.
- **Self-attested** — things Sentinels can never test passively without violating its own no-attack rule (e.g. "is rate limiting configured", "is authentication properly secured"). Sentinels simply asks the developer, honestly, rather than pretending to know.

Readiness score & deployment status

Readiness score — the percentage of Auto-tier items that pass. Self-attested and inferred items are deliberately excluded from this number so it can't be gamed by leaving hard questions unanswered.

Deployment status — one of three plain-English verdicts:

- **Blocked** — at least one Auto-tier item marked "blocking" has failed (examples: HTTPS not enforced, a live .env exposed, a secret committed to the repo, an unpatched critical dependency).
- **Caution** — nothing blocking, but something Auto or Inferred is still failing or warning.
- **Ready** — every Auto and Inferred item passes.

The website side ships 10 Auto rules, 2 Inferred, and 4 Self-attested. The repo side ships 9 Auto rules (3 of them blocking: secrets committed, .env not gitignored, a critical CVE present), 4 Inferred, and 4 Self-attested (secrets rotated, branch protection, two-factor auth, required code review).

Part 5

The AI Layer

Sentinels uses Groq's free-tier API with the model `openai/gpt-oss-20b`, chosen specifically because it's a reasoning model on a free tier, and a low reasoning-effort setting keeps token usage — and cost — small for these short, structured tasks. Every single call to it goes through one shared low-level function, and that function's contract is the whole reason the AI layer can never bring the app down: **on any failure at all — missing key, network error, rate limit, an unexpected response shape — it returns nothing, and never raises.** This is the one place "AI is optional" is centrally enforced, rather than every caller needing its own try/except.

The three AI features

1. Plain-language summary

Runs automatically at the end of every scan. Takes the score, grade, and findings, and writes a short paragraph leading with the actual worst real problem — not a generic "looks pretty good" filler. With no API key configured, this field is simply an empty string; every other part of the report (score, grade, findings, checklist, readiness) is computed exactly the same either way.

2. "Fix with AI"

An on-demand, cached suggestion for one specific finding: why the problem exists, its real security impact, a conceptual (never a working, copy-pasteable) description of how it could be exploited, a recommended fix, best practices, and framework-specific example code. For a repository finding, the example is an actual before/after code diff rather than a server-config block, since the fix lives in source code, not a live server. Results are cached in the database so the same fix isn't regenerated (and re-billed) every time the page loads; a regenerate option bypasses the cache on request. With no API key, this returns a clean "unavailable" response — never a server error.

3. Chat

A simple, no-frills question-and-answer window scoped to one finished scan. There's no vector database or retrieval step — the entire scan's findings and checklist are compressed into the system prompt directly, along with the last few turns of conversation history, and handed straight to the model. With no API key, the chat tab shows a clear "unavailable, set GROQ_API_KEY" screen instead of a broken input box.

Why this design matters

Every AI feature is additive. A developer who never configures an API key gets a fully complete, fully scored, fully explained report — they just don't get the plain-language narration or the one-click fix. This was a deliberate project rule from day one, and it means Sentinels never becomes unusable just because a free-tier API happens to be down, rate-limited, or unconfigured.

Part 6

Data Model & Persistence

Every scanner agent, regardless of type, produces a list of one shared shape — a `Finding` — so the orchestrator, the scorer, the AI layer, and the report renderer all speak exactly one language.

The core shapes

Model	Holds
Finding	id, title, category, severity, status (pass/warn/fail), OWASP tag, evidence, description, remediation, which agent produced it, structured evidence items, and — for repo scans only — the file path and line number
AgentResult	One agent's findings, how long it took, and an error field if it failed
ScanReport	Everything: id, target url, target_type (url/repo), score, grade, AI summary, severity counts, every finding, every agent's result, readiness score, deployment status, and the full checklist
ChecklistItem	One row: which tier, current state, plain-English explanation, suggested fix
RepoFileEntry	One file in a repo's tree: path, size, guessed language, and how many findings live in it — backs the file-tree browser

Persistence: SQLite, no ORM

Sentinels uses Python's built-in sqlite3 module directly rather than an ORM — a deliberate choice, since the entire query surface across the whole project is only around a dozen distinct queries, which doesn't justify the dependency and abstraction cost of an ORM. One connection is opened and closed per call rather than held open, since SQLite connections aren't safe to share across threads and FastAPI can dispatch requests onto different ones.

How the schema grew — the migration history

The database schema has grown through seven versions, applied automatically and safely on every startup — a fresh database gets every migration, an existing one only gets whatever it's missing:

Version	What it added
V1	scans, agent_runs, and findings tables — the original core
V2	evidence_items — structured, labelled proof behind each finding
V3	checklist_items — the three-tier readiness system
V4	fix_suggestions — the cached "Fix with AI" results
V5	chat_messages — the chatbot's conversation history
V6	target_type on scans, plus file_path/line on findings — the column that let one schema serve both website and repo scans, backfilling every old row automatically

Version	What it added
V7	repo_files — backs the file-tree browser (the most recently shipped feature)

This additive-only migration style is a running theme in the project: new capabilities are added as new, nullable/defaulted columns and new tables, never by changing what an existing row means. Every scan taken before the V6 migration, for example, reads back today exactly as `target_type="url"` — which is exactly what it always was.

Part 7

Report Export

A finished report can be exported in three formats, all built from the same underlying report data:

Format	How it's built
PDF	The report is first rendered as a standalone, dark-themed HTML document (score ring as inline SVG, findings grouped by category, checklist, agent log, any cached AI fixes). Playwright then drives a real headless Chromium browser to "print" that HTML to a PDF — the same mechanism as pressing Ctrl+P in a browser.
JSON	The report object, dumped essentially as-is — a clean machine-readable export.
Markdown	A plain-text version using the same category grouping as the PDF, for anyone who wants the report in a plain document instead.

One real Windows-specific engineering detail worth knowing: the backend's live-reload server forces a particular event-loop type that Playwright's process-launching can't use directly, so PDF generation on Windows runs on its own dedicated event loop in a background thread rather than trusting the app's default one.

Part 8

The API Surface

The backend is a single FastAPI application. Cross-origin requests are restricted explicitly to the frontend's own address — never a wildcard — specifically so no other website could quietly use a visitor's browser to drive scan traffic through Sentinels' backend.

Group	Endpoints
Meta	GET / · GET /health
Website scan	POST /scan · GET /scan/stream (live progress via Server-Sent Events)
Repo scan	POST /repo/scan · GET /repo/stream
Agent metadata	GET /agents · GET /repo/agents
Scan records	GET /scans · GET /scans/{id} · GET /scans/{id}/agents/{name} · GET /scans/{id}/files · DELETE /scans/{id}
Checklist	GET /scans/{id}/checklist · POST /scans/{id}/checklist/{item} (only self-attested items are writable)
AI fixes	POST /scans/{id}/findings/{key}/fix
Chat	POST /scans/{id}/chat · GET /scans/{id}/chat
Export	GET /export/formats · GET /scans/{id}/export/{format}

Live progress uses Server-Sent Events rather than WebSockets — a simpler, one-directional protocol that's a perfect fit here since the browser only ever needs to *receive* updates, never send anything mid-scan. Each connection streams one `event: agent` message per agent, in the real order they actually finish (not a fixed order), followed by one final `event: done` carrying the complete report.

Part 9

Frontend & User Journey

The page structure

Route	What it shows
/	Landing page — an animated intro sequence ending in a two-path chooser: scan a website, or scan a GitHub repo
/url, /repo	The launcher for each scan type — one input, one button
/scan/{id}	The finished report: score ring, severity counts, deployment badge, AI summary, a grid of the five agents, a "Download PDF" button
/scan/{id}/agents/{name}	One agent's full detail — its purpose, exactly what it checks, every finding with evidence, and "Fix with AI" on each
/scan/{id}/checklist	The three-tier readiness checklist, with the readiness score and deployment badge
/scan/{id}/files	Repo scans only — a collapsible file tree, each file badged with its finding count, click through to see that file's findings
/scan/{id}/chat	Ask follow-up questions about this specific scan

The live-scan experience

Submitting a URL or repo opens a live streaming connection and shows five glass-panel tiles, one per agent, each one visually "materializing" the instant its own result actually lands — not on a fixed timer, and not all five at once. The agent list itself is fetched from the backend rather than hard-coded, so a sixth agent would show up automatically with zero frontend changes. Once the scan finishes, the browser is sent to the permanent report page — which is already fully saved server-side, so the destination page loads cleanly and survives a hard refresh, a closed tab, or coming back a week later.

The complete user journey

- 1. Land on the homepage, scroll through a short animated intro.
- 2. Choose website or GitHub repo.
- 3. Paste a URL or repo link, submit.
- 4. Watch the five relevant agents complete live, in real time.
- 5. Arrive at the full report: score, grade, plain-English summary, and a clear list of what's wrong.
- 6. Drill into any specific agent for full detail, or the checklist for a deployment-readiness view, or (for a repo) the file tree.
- 7. Ask the built-in chat a follow-up question, or ask the AI to generate a concrete fix for any single finding.
- 8. Download the whole thing as a PDF to share or keep.

Part 10

How It Was Built

Sentinels was deliberately built as a learning project as much as a product — every single piece of new code was written alongside a plain-English note explaining the concept behind it, in build order. The project shipped in three major, fully-completed phases.

v1 — the core engine (18 achievements)

Built the foundational shape of the whole system against one agent first (Headers), then proved it generalizes by adding the remaining four website agents and running them genuinely concurrently — the project's own measurements showed roughly 1.3–1.5 seconds running all five agents in parallel, versus about 2.6 seconds running the same five one after another. From there: the AI analyst layer with graceful degradation, the first Next.js frontend, live progress via Server-Sent Events, Playwright-based PDF export, and a final "hand the repository to a stranger and see if it runs clean" check.

v2 — persistence, checklist, AI fixes, export (19 milestones)

The single biggest structural change in this phase: v1 was completely stateless — a finished scan existed only in the browser's memory and vanished on refresh. v2's very first milestone was adding SQLite persistence, which every later feature (a scan history dashboard, per-agent detail pages, cached AI fixes, chat memory) then depended on. This phase also had to resolve a real design tension head-on: several requested "deployment checklist" items (is input validation in place? is rate limiting configured?) simply cannot be tested passively without violating the project's own no-active-testing rule. The resolution was the three-tier auto / inferred / self-attested checklist system described in Part 4 — rather than faking certainty, Sentinels is explicit about exactly how confident it actually is in each answer.

v3 — GitHub repository scanning (12 milestones, R1–R12)

Adds the second scan type end-to-end, built for what the project's own planning documents call the "vibe coder" — someone who shipped a real project with an AI coding assistant's help and has no security background to know what they got wrong. The guiding architectural principle was "extend, don't fork": the scoring engine, the checklist evaluator, the entire AI layer, the export and storage systems, and most of the frontend's report pages are reused completely unchanged — only the repository fetcher and the five new repo-specific agents are genuinely new code. Two new rules were added to the project's own non-negotiables at this stage: never execute anything from a scanned repository, and never echo a discovered secret. The final milestone, a collapsible file-tree browser for repo scans, shipped most recently — all twelve milestones of this phase are now complete.

The build discipline

Across all three phases, the same three rules were followed without exception: build one small, independently-verifiable milestone at a time, and never start the next one until the current one is actually verified working end-to-end; write one plain-English learning note per milestone, alongside the code, not after; and never let a feature ship that would require sending harmful traffic to accomplish its goal.

Part 11

Presentation Q&A — Ideathon Answers

The following answers the standard pitch-deck questions the presentation template asks for (Introduction, The Big Problem, Our Solution, Why Now, Market Opportunity, Value Proposition, Competitive Landscape, Conclusions), written from everything documented above.

Introduction

Sentinels is a passive security auditor for websites and GitHub repositories. A user pastes a URL or a public repo link; within about a minute, they get back a 0–100 security score, an A–F grade, and a plain-English breakdown of exactly what's wrong and how to fix it — powered by five concurrently-running scanner agents and an optional AI layer, and built on a strict rule that it never sends anything that could actually harm the target.

The Big Problem

AI coding assistants have made it possible for anyone to ship a working website or application in an afternoon, with essentially no security background required. "Working" and "secure" are not the same thing, and AI assistants overwhelmingly optimize for the former — a generated app can run perfectly while leaving a `.env` file publicly reachable, missing every security header, or having a real API key committed straight into the git history. The people this affects most — students, hackathon teams, solo indie developers, small teams without a dedicated security hire — have no fast, free, and safe way to find out. The existing options are a bad fit at every level: real penetration-testing tools require expertise most beginners simply don't have and can be legally risky to point at a system you don't fully control; a professional security audit costs money and takes days a hackathon team doesn't have; and most free tools only check one thin slice of the problem (headers only, or dependencies only) rather than giving one complete picture.

Our Solution

Sentinels closes that gap with a single design goal: security feedback should be as easy to get as pasting a link. Paste a website URL or a GitHub repo, and in under a minute get a deterministic score, a clear grade, and a full breakdown across five specialized checks — with an AI layer that explains findings in plain English and can generate an actual code-level fix on request. Because every single check is passive by design, there is zero risk of Sentinels ever damaging the thing it's scanning, which means it's genuinely safe to point at your own in-progress project the moment you want a second opinion — no legal grey area, no need to ask permission from anyone, no risk of knocking anything over.

Why Now

- **The AI-coding boom changed who's shipping code.** Tools like GitHub Copilot, Cursor, and Claude Code have put full application development in the hands of people with zero security training, at a scale that didn't exist even two years ago — and that population is growing fast, not slowing down.

- **Security is the part AI assistants are worst at.** They optimize for "does it run," not "is it safe," so the exact population now shipping the most code is also the population least equipped to catch what got missed.
- **Cheap, capable LLM inference finally makes this economical.** Groq's free tier makes it realistic to add genuine plain-language explanation and fix generation to a completely free tool — this simply wasn't cost-viable to offer for free even a couple of years ago.
- **Public leaks are a proven, recurring, real cost.** Committed API keys and exposed .env files are one of the most common real-world incident types on public GitHub today — this project's own history contains exactly that incident, which is precisely the class of mistake Sentinels is built to catch before it happens to someone else.

Market Opportunity

The primary audience is anyone building and shipping software without a dedicated security function behind them: student developers and coding-bootcamp learners, hackathon teams (who need a fast credibility check before a demo, not a week-long audit), solo indie hackers and early-stage startup founders, and small development teams that haven't yet hired security expertise. This segment is large and structurally growing — it scales directly with the population of AI-assisted developers, which is one of the fastest-growing groups in software today. A natural secondary audience is educational: instructors and bootcamps could use Sentinels as a teaching tool, letting students see concretely, on their own project, what a real security issue actually looks like.

Value Proposition

- **Zero friction.** No signup, no install, no configuration — one link in, one report out, in under a minute.
- **No security expertise required.** Every finding is explained in plain English with an AI layer that can generate an actual fix, not just a jargon-filled list of what's wrong.
- **Provably safe to use on anything.** The passive-only guarantee means there is no legal or technical risk in scanning your own live site or repo, unlike active-scanning security tools.
- **Covers both halves of a real project.** Most comparable tools check either a live deployment or a source repository — Sentinels checks both, from one product, with one consistent report format.
- **Deterministic and trustworthy.** The score is pure rule-based arithmetic, not an AI guess — scan the same thing twice and get the same number, every time.
- **Actionable, not just diagnostic.** "Fix with AI" turns a finding into a concrete recommended fix and, for code, an actual before/after diff — the report tells you what to do next, not just what's wrong.

Competitive Landscape

Competitor	Gap Sentinels fills
OWASP ZAP / Burp Suite	Powerful, but require real security expertise to use correctly, actively attack the target (legally risky against anything you don't fully own), and have a steep learning curve — not remotely beginner-friendly.
Mozilla Observatory / SecurityHeaders.com	Passive and free like Sentinels, but website-only (no repo scanning), header/TLS-focused only, and offer no AI explanation or fix generation — you get a list, not an understanding.
Snyk / GitHub Dependabot / GitGuardian	Excellent at dependency and secret scanning specifically, but repo-only (no live-site scanning), typically require signup, and are positioned/priced for teams and enterprises rather than a solo student or hacker.

Competitor	Gap Sentinels fills
Manual security audits / freelance pentesters	Thorough, but slow (days to weeks) and expensive — completely impractical for a hackathon deadline or a side project's weekend check-in.

Sentinels' actual differentiation isn't any single check — it's being the only option that combines passive live-website scanning, GitHub repository scanning, and plain-language AI explanation, in one free, signup-free, minute-long flow built specifically for people without a security background.

Conclusions

Sentinels addresses a real and growing gap: the fastest-growing population of developers today is also the one least equipped to catch its own security mistakes, and the tools that exist either demand expertise they don't have, cost money and time they don't have, or only look at half the picture. Sentinels closes that gap with one link, one minute, one clear score, and a plain-English explanation of exactly what to fix and why — built entirely on checks that can never cause harm. The product is feature-complete today across both scan types (website and GitHub repository), the entire scoring and checklist system is deterministic and trustworthy by design, and every AI feature degrades gracefully rather than being a single point of failure. It's ready to demo, ready to use on a real project today, and built on a foundation — deterministic scoring, passive-only checks, graceful AI degradation — that scales cleanly to whatever comes next.